# IT3708 - Bio-Inspired Artificial Intelligence

***Ole J. Mengshoel***
ole.j.mengshoel@ntnu.no

***Professor, Department of CS, NTNU, Trondheim***

***Adjunct Faculty, Department of ECE, CMU, Silicon Valley***

***March 3rd, Spring 2020 Semester***

***THIS IS A VERSION FOCUSED ON THE TEST PART OF THE LECTURE ONLY, RELEASED ON 12.5.2025 AS PART OF EXAM PREPARATIONS IN 2025.***

# Question 1

☐ 1. **Multiple Choice: Among the following options, which op...**

Points: 1

| | |
|---|---|
| Question | Among the following options, which option is not considered a "pillar of evolution"? |
| Answer | Population |
| | Diversity |
| | Heredity |
| | Constraints |
| | Selection |

# Question 1

☐ 1. **Multiple Choice: Among the following options, which op...**

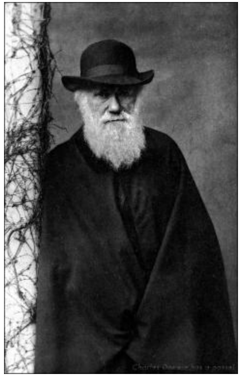
Points: 1

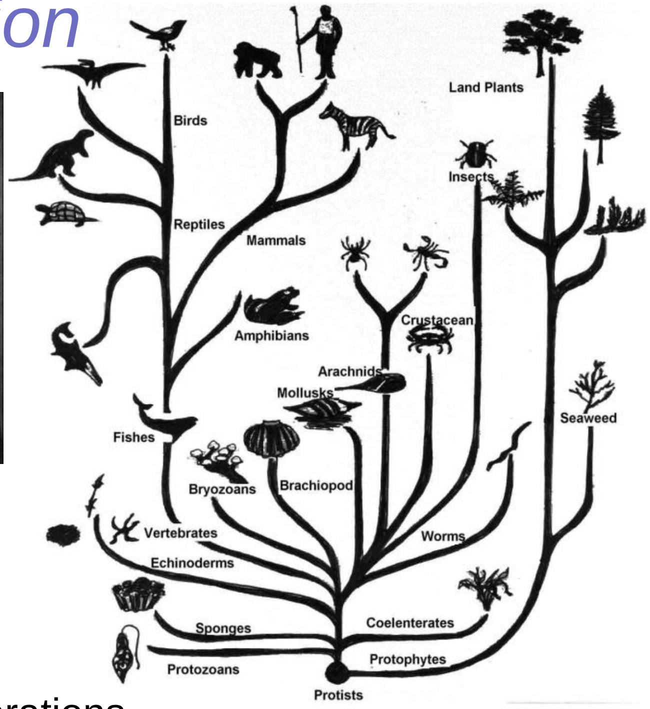
| | |
|---|---|
| Question | Among the following options, which option is not considered a "pillar of evolution"? |
| Answer | Population<br>Diversity<br>Heredity<br>✅ Constraints<br>Selection |

# The 4 Pillars of Evolution

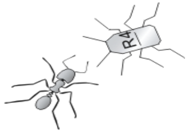
All species derive from common ancestor

Charles Darwin, 1859
*On the Origins of Species*






**Population**
Group of several individuals
**Diversity**
Individuals have different characteristics
**Heredity**
Characteristics are transmitted over generations
**Selection**
- Individuals make more offspring than the environment can support
- Better at food gathering = better at surviving = make more offspring



# Question 2

☐ **2. Multiple Choice: Among the following operations, which...**

Points: 1

| | |
|---|---|
| **Question** | Among the following operations, which is not part of an evolutionary algorithm's generation cycle? |
| **Answer** | Mutation<br>Evaluation<br>Initialization<br>Crossover<br>Selection<br>Reproduction |

# Question 2

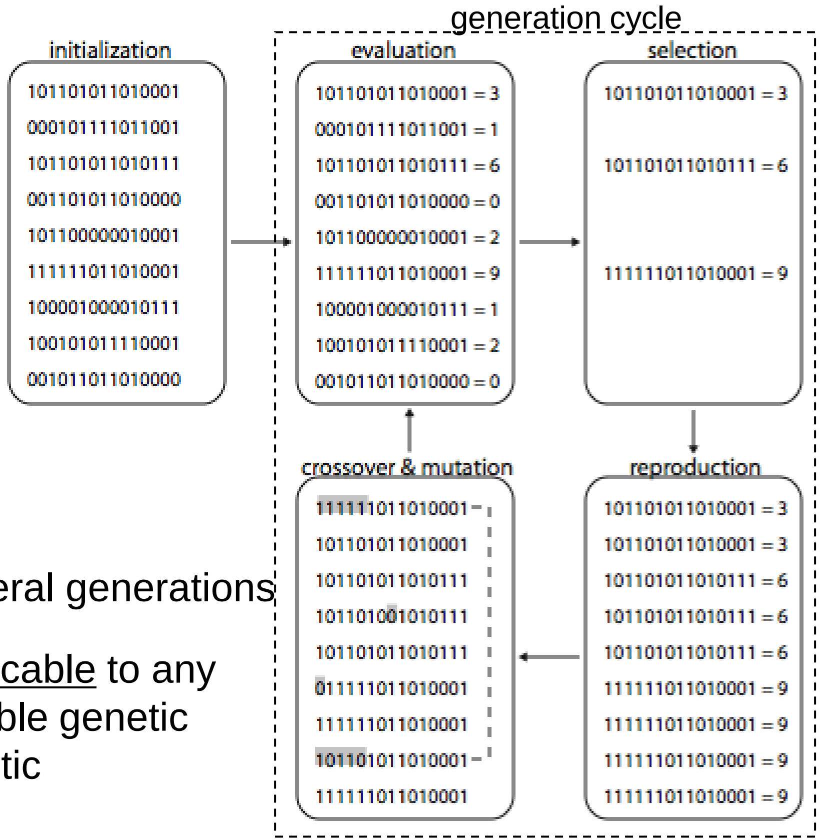
☐ **2. Multiple Choice: Among the following operations, which...**

Points: 1

| | |
|---|---|
| Question | Among the following operations, which is not part of an evolutionary algorithm's generation cycle? |
| Answer | Mutation |
| | Evaluation |
| | ✅ Initialization |
| | Crossover |
| | Selection |
| | Reproduction |

# Evolutionary Algorithm

- Devise genetic representation
- Build a population
- Design a fitness function
- Choose selection method
- Choose crossover & mutation
- Choose data analysis method

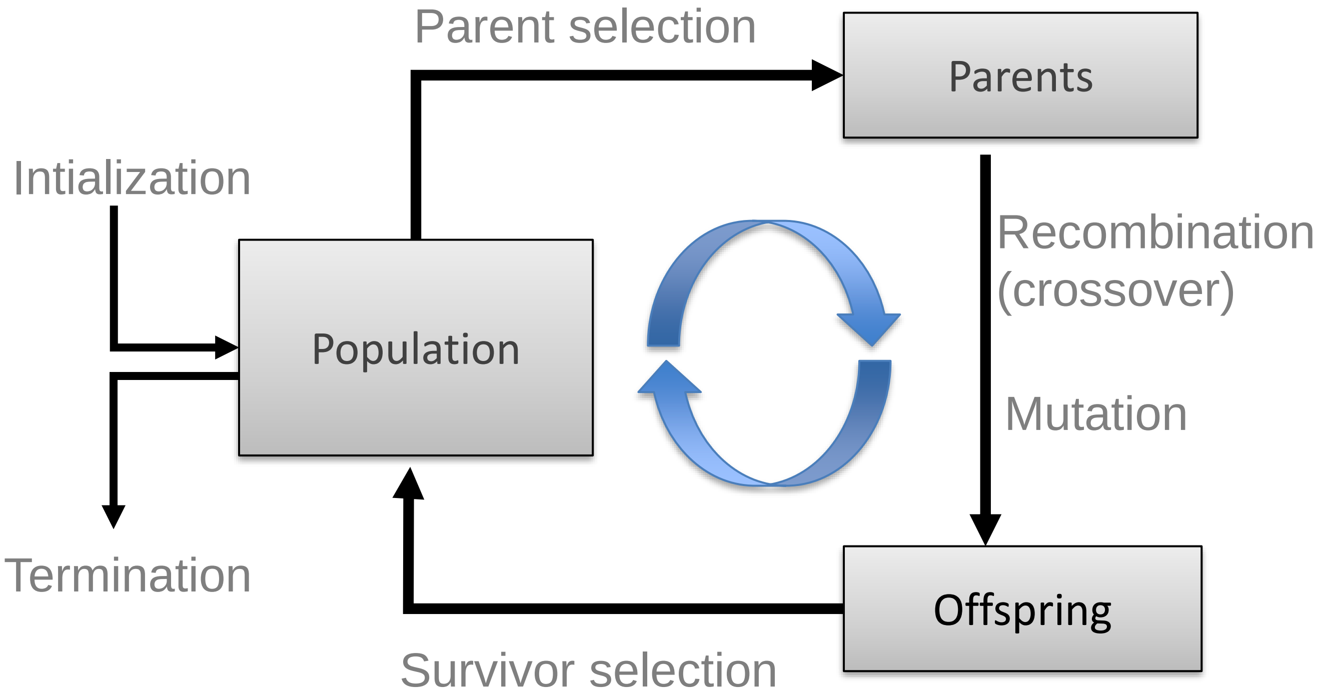



<u>Repeat</u> generation cycle until:

- maximum fitness value is found
- solution found is good enough
- no fitness improvement for several generations

Evolutionary algorithms are <u>applicable</u> to any problem domain as long as suitable genetic representation, fitness, and genetic operators are chosen.

# Scheme of an EA:
# General scheme of EAs



# Question 3

☐ 3. **Multiple Choice: In which type of genetic algorithm (G...**

Points: 1

| | |
|---|---|
| Question | In which type of genetic algorithm (GA) is one individual generated per generation, replacing one individual in the population? |
| Answer | Crowding GA |
| | Generational GA |
| | Steady-state GA |
| | Messy GA |
| | Elitist GA |

# Question 3

☐ **3. Multiple Choice: In which type of genetic algorithm (G...** Points: 1

| | |
|---|---|
| Question | In which type of genetic algorithm (GA) is one individual generated per generation, replacing one individual in the population? |
| Answer | Crowding GA |
| | Generational GA |
| | ✅ Steady-state GA |
| | Messy GA |
| | Elitist GA |

# Population Management Models: Introduction

- Two different population management models exist:
  - *Generational genetic algorithm* (GGA)
    - each individual survives for exactly one generation
    - the entire set of parents is replaced by the offspring
  - *Steady-state genetic algorithm* (SSGA)
    - one offspring is generated per generation
    - one member of population replaced
- Most EAs use either the GGA or the SSGA mode
  - The simple genetic algorithm (*SGA*) of Goldberg uses GGA
  - Whitley's *GENITOR* is a steady-state GA - SSGA
- Generation Gap (GG)
  - The proportion of the population replaced at each iteration
- How parameter GG varies:
  - For GGA: GG = 1
  - For SSGA: GG = 1/pop_size

# Question 4

☐ 4. **Multiple Choice: What is a potential unfortunate conse...**

Points: 1

| | |
|---|---|
| Question | What is a potential unfortunate consequence of too high selection pressure in an evolutionary algorithm that is performing optimization? |
| Answer | The population converges to individuals at or close to a local optimum that is not a global optimum. |
| | The fitness of the population does not increase much. |
| | All individuals in the population converge to be exactly alike, at a local optimum that is not a global optimum. |
| | All individuals in the population converge to be exactly alike, at a global optimum. |
| | The population converges to individuals at or close to a global optimum. |

# Question 4

☐ 4. **Multiple Choice: What is a potential unfortunate conse...**

Points: 1

| | |
|---|---|
| Question | What is a potential unfortunate consequence of too high selection pressure in an evolutionary algorithm that is performing optimization? |
| Answer | ✅ The population converges to individuals at or close to a local optimum that is not a global optimum. |
| | The fitness of the population does not increase much. |
| | All individuals in the population converge to be exactly alike, at a local optimum that is not a global optimum. |
| | All individuals in the population converge to be exactly alike, at a global optimum. |
| | The population converges to individuals at or close to a global optimum. |

# Selection and Selection Pressure: Summary

- Intuition: Selection plays a role in creating selection pressure in the Evolutionary Algorithm
- *Selection pressure*:
  - Can be defined mathematically as [Simon 2013, p. 199] as:
  - $\Phi$ = Pr(selection of best fit) / Pr(selection of average fit)
  - But can also be understood more intuitively
- *Too high selection pressure*: The problem of premature convergence, the population converges to individuals at a local optimum
- *Too low selection pressure*: The problem of too little convergence, the fitness of the population doesn't increase much – optima are not found before the algorithms terminates
- [Simon 2013, p. 199]: «If the selection method is overly biased towards selecting fit individuals, then the population may converge too quickly to a uniform solution […]. However, if a selection methods is not biased strongly enough towards fit individuals, then the EA may not be able to properly exploit the information that is present in the most fit individuals»

# Question 5

☐ 5. **Multiple Choice: Which of the following is not conside...**

Points: 1

| | |
|---|---|
| Question | Which of the following is not considered a space in which to preserve diversity in an evolutionary algorithm? |
| Answer | Algorithm space |
| | Probability space |
| | Genotype space |
| | Phenotype space |

# Question 5

☐ 5. **Multiple Choice: Which of the following is not conside...**

Points: 1

| | |
|---|---|
| Question | Which of the following is not considered a space in which to preserve diversity in an evolutionary algorithm? |
| Answer | Algorithm space |
| | ✓ Probability space |
| | Genotype space |
| | Phenotype space |

# Approaches for Preserving Diversity: Introduction (1/2)

Different spaces for preserving diversity:

- Genotype space
  - Set of representable solutions
- Phenotype space
  - The end result
  - Neighbourhood structure may bear little relation with genotype space
- Algorithmic space
  - Equivalent of the geographical space on which life on earth has evolved
  - Structuring the population of candidate solutions

# Question 6

☐ **6. Multiple Choice: Which of the following is not conside...**

Points: 1

| | |
|---|---|
| **Question** | Which of the following is not considered a diversity-preserving genetic algorithm? |
| **Answer** | Deterministic crowding genetic algorithm<br>Fitness sharing genetic algorithm<br>Probabilistic crowding genetic algorithm<br>Simple genetic algorithm<br>Island model parallel genetic algorithm |

# Question 6

☐ **6. Multiple Choice: Which of the following is not conside...** Points: 1

| | |
|---|---|
| **Question** | Which of the following is not considered a diversity-preserving genetic algorithm? |
| **Answer** | Deterministic crowding genetic algorithm |
| | Fitness sharing genetic algorithm |
| | Probabilistic crowding genetic algorithm |
| | ✅ Simple genetic algorithm |
| | Island model parallel genetic algorithm |

# Crowding in GAs

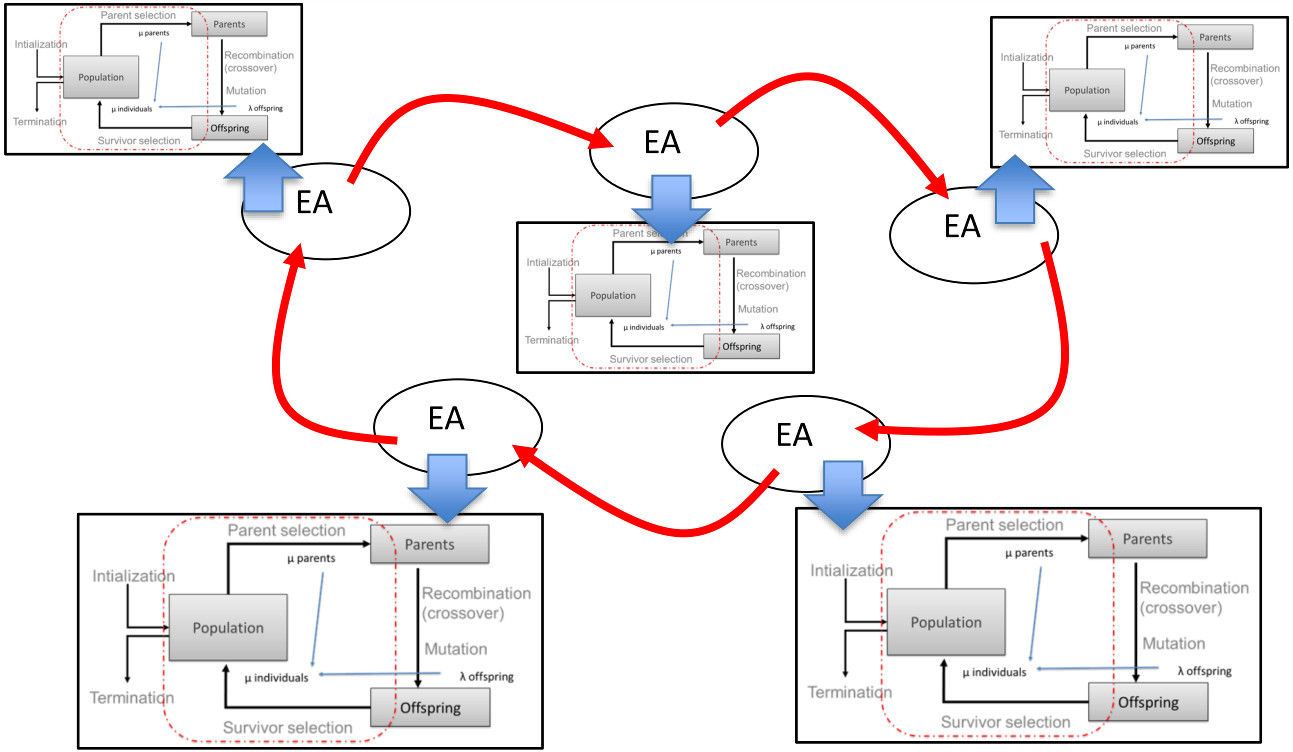
- Crowding applies in the survival stage of GAs; phases:
  - *Grouping*: individuals are grouped (paired), often but not necessarily with their offspring, according to a similarity metric
  - *Replacement (competition)*: for each group (pair), one or more winners are picked – remain in the population
- Crowding variants in evolutionary algorithms:
  - Preselection [Cavicchio, 70]
  - Crowding factor [DeJong, 75]
  - Simulated annealing [Kirkpatrick et al., 83]
  - Gene-invariant GA [Culberson, 92]
  - Deterministic crowding [Mahfoud, 95]
  - Parallel recombinative SA [Mahfoud & Goldberg, 95]
  - Restricted Tournament Selection [Harik, 95]
  - Probabilistic crowding [Mengshoel & Goldberg, 99]
  - Generalized crowding [Galan & Mengshoel, 10]
  - Adaptive generalized crowding [Mengshoel et al., 13]
  - …

# Explicit Approaches for Preserving Diversity: Fitness Sharing (1/2)

- Restricts the number of individuals within a given niche by "sharing" their fitness:
  - Niche fitness is a shared resource – think sheep eating grass
  - Individuals are allocated to a niche in proportion to the niche fitness
- Need to set the size of the niche $\sigma_{share}$ in either genotype or phenotype space
  - Here we assume genotype space
- Run EA as normal but after each generation set

$$f'(i) = \frac{f(i)}{\sum_{j=1}^{\mu} sh(d(i,j))} \qquad sh(d) = \begin{cases} 1-(d/\sigma_{\text{share}})^{\alpha} & d \le \sigma_{\text{share}} \\ 0 & otherwise \end{cases}$$

# Implicit Approaches for Preserving Diversity: "Island" Model Parallel EAs (1/3)




: Periodic migration of individual solutions between populations

# Question 7

☐ 7. **Multiple Choice: In the 8-queens problem, as discussed...**

Points: 3

| | |
|---|---|
| Question | In the 8-queens problem, as discussed in class, there are three types of constraints: horizontal, vertical and cross. To solve the 8-queens problem, we consider a genetic algorithm using a permutation representation. In this permutation representation, the i-th position in the chromosome represents the i-th column on the chessboard and the value there represents a row on the chessboard. Consider, for example, the chromosome (1, 3, 5, 2, 6, 4, 7, 8). Here, the "2" in the 4th position of the chromosome means there is a queen in the 2nd row for the 4th column of the chessboard.<br><br>Mutation and crossover work as follows, as discussed in class: Mutation takes, for example, (1, 3, 5, 2, 6, 4, 7, 8) and creates (3, 1, 5, 2, 6, 4, 7, 8). Crossover takes, for example, parents (1, 3, 5, 2, 6, 4, 7, 8) and (8, 7, 6, 5, 4, 3, 2, 1), and creates children (1, 3, 5, 4, 2, 8, 7, 6) and (8, 7, 6, 2, 4, 1, 3, 5). The fitness function takes care of remaining constraint violations, if any, by counting the number of constraints violated. Fitness of an individual is inversely proportional to the number of constraints violated.<br><br>How can we describe the constraint handling of the genetic algorithm in this 8-queens problem? |
| Answer | All constraints are handled by the fitness function<br><br>The cross and vertical constraints are handled by the fitness function, while the horizontal constraints are handled by other GA operators<br><br>The vertical constraints are handled by the fitness function, while the cross and horizontal constraints are handled by other GA operators<br><br>The cross and horizontal constraints are handled by the fitness function, while the vertical constraints are handled by other GA operators<br><br>The horizontal constraints are handled by the fitness function, while the cross and vertical constraints are handled by other GA operators<br><br>The cross constraints are handled by the fitness function, while the horizontal and vertical constraints are handled by other GA operators |

# Question 7

☐ 7. **Multiple Choice: In the 8-queens problem, as discussed...**

Points: 3

**Question**

In the 8-queens problem, as discussed in class, there are three types of constraints: horizontal, vertical and cross. To solve the 8-queens problem, we consider a genetic algorithm using a permutation representation. In this permutation representation, the i-th position in the chromosome represents the i-th column on the chessboard and the value there represents a row on the chessboard. Consider, for example, the chromosome (1, 3, 5, 2, 6, 4, 7, 8). Here, the "2" in the 4th position of the chromosome means there is a queen in the 2nd row for the 4th column of the chessboard.

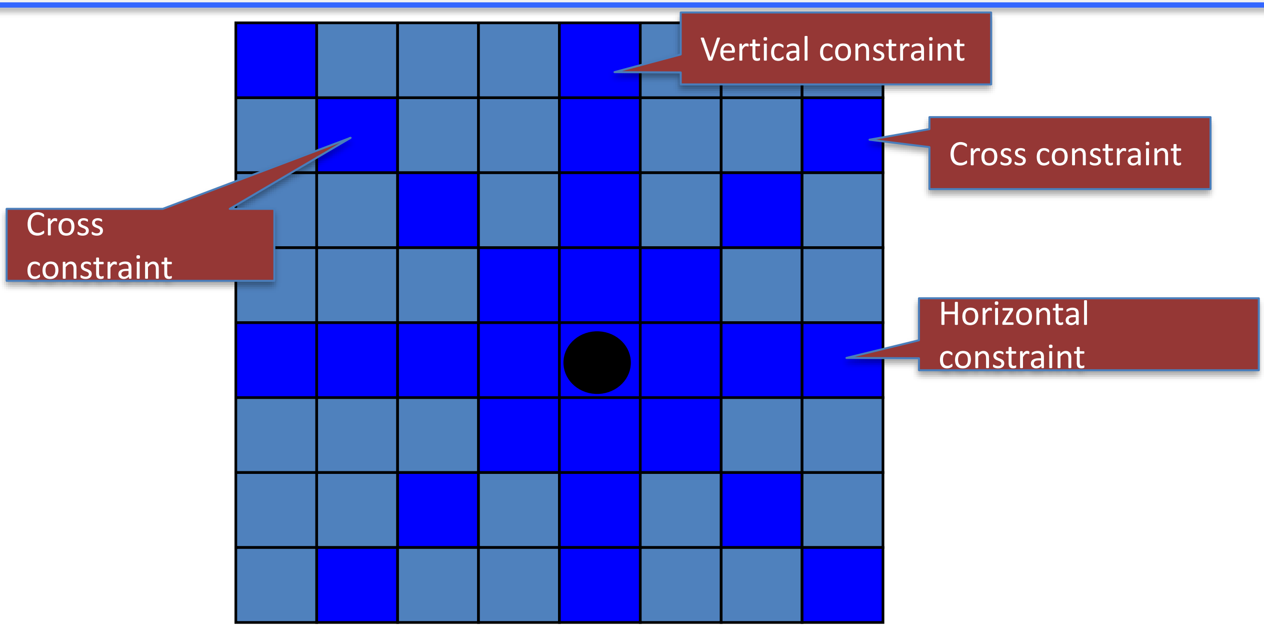
Mutation and crossover work as follows, as discussed in class: Mutation takes, for example, (1, 3, 5, 2, 6, 4, 7, 8) and creates (3, 1, 5, 2, 6, 4, 7, 8). Crossover takes, for example, parents (1, 3, 5, 2, 6, 4, 7, 8) and (8, 7, 6, 5, 4, 3, 2, 1), and creates children (1, 3, 5, 4, 2, 8, 7, 6) and (8, 7, 6, 2, 4, 1, 3, 5). The fitness function takes care of remaining constraint violations, if any, by counting the number of constraints violated. Fitness of an individual is inversely proportional to the number of constraints violated.

How can we describe the constraint handling of the genetic algorithm in this 8-queens problem?

**Answer**

- All constraints are handled by the fitness function
- The cross and vertical constraints are handled by the fitness function, while the horizontal constraints are handled by other GA operators
- The vertical constraints are handled by the fitness function, while the cross and horizontal constraints are handled by other GA operators
- The cross and horizontal constraints are handled by the fitness function, while the vertical constraints are handled by other GA operators
- The horizontal constraints are handled by the fitness function, while the cross and vertical constraints are handled by other GA operators
- ✅ The cross constraints are handled by the fitness function, while the horizontal and vertical constraints are handled by other GA operators

# Example:
# The 8-queens (CSP) problem (2/2)



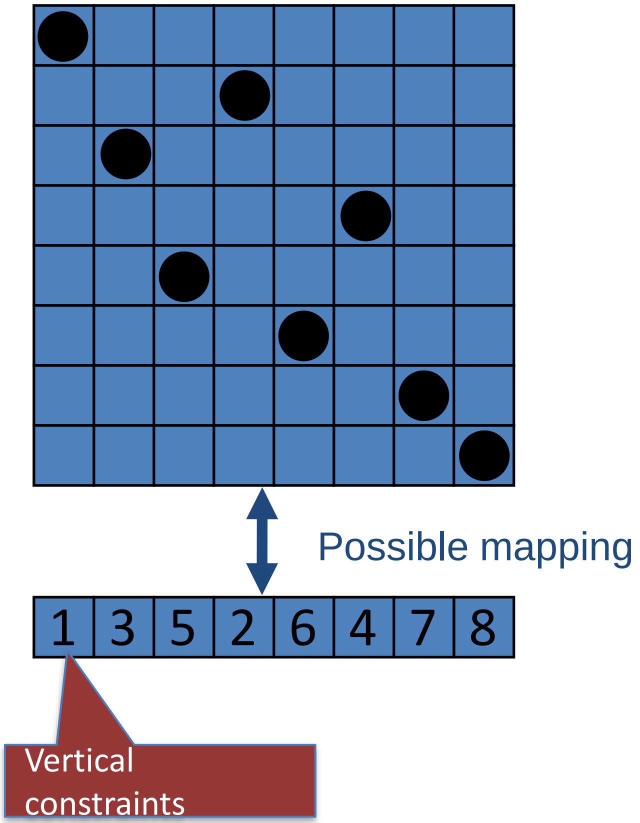

Place 8 queens on an 8x8 chessboard in
such a way that they cannot check each other

# The 8-queens problem: Representation

Phenotype:
a board configuration

Horizontal constraints

Genotype:
a permutation of
the numbers 1–8



# The 8-queens problem: Fitness evaluation

- Penalty of one queen: the number of queens she can check
- Penalty of a configuration: the sum of penalties of all queens
- Note: penalty is to be minimized
- Fitness of a configuration: inverse penalty to be maximized

Cross constraints

# Question 8

☐ 8. **Multiple Choice: Suppose that we have a problem in whi...** Points: 1

| | |
|---|---|
| Question | Suppose that we have a problem in which a genetic algorithm with a permutation representation is being used. For example, the problem may be the 8-queens problem or the traveling salesman problem (TSP). Which of the following crossover operators would perform poorly for this problem? |
| Answer | Edge recombination crossover<br>Partially mapped crossover<br>Order 1 crossover<br>One-point crossover |

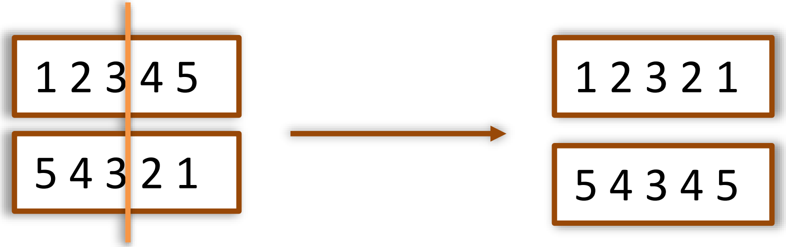
# Question 8

☐ 8. **Multiple Choice: Suppose that we have a problem in whi…** Points: 1

| | |
|---|---|
| **Question** | Suppose that we have a problem in which a genetic algorithm with a permutation representation is being used. For example, the problem may be the 8-queens problem or the traveling salesman problem (TSP). Which of the following crossover operators would perform poorly for this problem? |
| **Answer** | Edge recombination crossover<br>Partially mapped crossover<br>Order 1 crossover<br>✔ One-point crossover |

# Permutation Representations: Crossover operators

- “Normal” crossover operators will often lead to inadmissible permutation solutions




- Many specialised operators have been devised which focus on combining order or adjacency information from the two parents

# Question 9

☐ 9. **Multiple Choice: Which of the following types of Marko...** Points: 1

| | |
|---|---|
| Question | Which of the following types of Markov chains is most commonly used to analyze hill climbing and genetic algorithms that operate in a search space of bitstrings? |
| Answer | Discrete state space, continuous time<br>Discrete state space, discrete time<br>Continuous state space, continuous time<br>Continuous state space, discrete time |

# Question 9

☐ 9. **Multiple Choice: Which of the following types of Marko...**

Points: 1

| | |
|---|---|
| **Question** | Which of the following types of Markov chains is most commonly used to analyze hill climbing and genetic algorithms that operate in a search space of bitstrings? |
| **Answer** | Discrete state space, continuous time<br>✔ Discrete state space, discrete time<br>Continuous state space, continuous time<br>Continuous state space, discrete time |

# Question 10

☐ **10. Multiple Answer: Which of the following options are tr...**

Points: 1

| | |
|---|---|
| Question | Which of the following options are true for a homogenous Markov chain? (This is a multiple answer question.) |
| Answer | The 1-step transition probability matrix can never change |
| | The 1-step transition probability matrix can change |
| | All entries in the 1-step transition probability matrix are greater than zero |
| | All entries in the 1-step transition probability matrix are greater than or equal to zero |

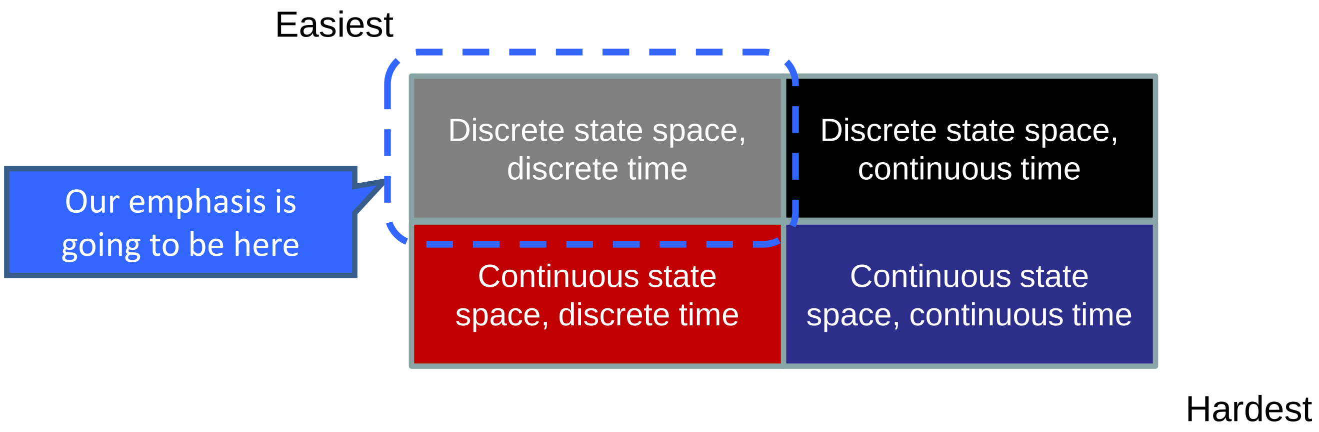
# Question 10

☐ **10. Multiple Answer: Which of the following options are tr...** Points: 1

| | |
|---|---|
| **Question** | Which of the following options are true for a homogenous Markov chain? (This is a multiple answer question.) |
| **Answer** | ✔ The 1-step transition probability matrix can never change |
| | The 1-step transition probability matrix can change |
| | All entries in the 1-step transition probability matrix are greater than zero |
| | ✔ All entries in the 1-step transition probability matrix are greater than or equal to zero |

# Markov Chains - Big Picture

We distinguish several types of Markov chains:




Other dimensions are also important. In this class, we focus on (time) <u>homogenous</u> Markov chains with <u>discrete time</u> (steps in algorithms are discrete) and <u>discrete state space</u> (our main focus is on discrete search spaces). Homogenous means that probabilities stay the same over time.

For the moment we are looking at stochastic processes for which both time parameter and state space are *discrete*. Furthermore, we assume that the state space is *finite*.

- $\{X_n, n \geq 0\}$;
- $S = \{1, 2, \ldots, N\}$;

**Questions that we like to answer:**

- What can we say about $X_{n+1}, X_{n+2}, X_{n+3}$, once we know $X_0, X_1, \ldots, X_n$? (short-term behaviour)
- Which part of the time a stochastic process spends in a certain state? (long-term behaviour)
- What are the average costs per period if in every state certain costs have to be paid? (e.g., inventory costs )

# Question 11

☐ 11. **Multiple Choice: Consider a population of size s, cons...**

Points: 2

| | |
|---|---|
| Question | Consider a population of size s, consisting of individuals that are bitstrings of length n. The population is initialized uniformly at random. Suppose that n = 5 and s = 4. What is the probability that the i-th bit of each individual in the population is the same for a given i? |
| Answer | Too little information is provided in the question to give an answer.<br>0,5<br>0,25<br>0,125<br>0,0625<br>0,03125 |

# Question 11

☐ **11. Multiple Choice: Consider a population of size s, cons...** Points: 2

| | |
|---|---|
| Question | Consider a population of size s, consisting of individuals that are bitstrings of length n. The population is initialized uniformly at random. Suppose that n = 5 and s = 4. What is the probability that the i-th bit of each individual in the population is the same for a given i? |
| Answer | Too little information is provided in the question to give an answer. |
| | 0,5 |
| | 0,25 |
| | ✔ 0,125 |
| | 0,0625 |
| | 0,03125 |

½*½*½*½*2 = 0,125. Multiply by 2 since it can happen in two ways, with 0s and 1s.

# Question 12

12. **Multiple Choice: Supppose that we have a genetic algor…**

Points: 3

| | |
|---|---|
| Question | Supppose that we have a genetic algorithm with a population of 10 individuals (x_1, ..., x_i, ..., x_10). Suppose that the fitness f of the i-th individual is f(x_i) = i, and that roulette wheel selection is used to select individuals for crossover. Selection is done with replacement. What is, approximately, the probablility that x_10 is selected to cross over with itself when two parents are selected from the popuation of 10? |
| Answer | 0% |
| | 0.9% |
| | 1.7% |
| | 2.1% |
| | 3.3% |
| | 9.5% |
| | 16.1% |
| | 18.2% |

# Question 12

12. **Multiple Choice: Supppose that we have a genetic algor…** Points: 3

| | |
|---|---|
| Question | Supppose that we have a genetic algorithm with a population of 10 individuals (x_1, ..., x_i, ..., x_10). Suppose that the fitness f of the i-th individual is f(x_i) = i, and that roulette wheel selection is used to select individuals for crossover. Selection is done with replacement. What is, approximately, the probablility that x_10 is selected to cross over with itself when two parents are selected from the popuation of 10? |
| Answer | 0% |
| | 0.9% |
| | 1.7% |
| | 2.1% |
| | ✓ 3.3% |
| | 9.5% |
| | 16.1% |
| | 18.2% |

Total fitness is (10+1) * 5 = 55.
Probability of one selection is 10/55 = 0.182.
Independence gives 0.182 * 0.182 = 0.33, or 3.3%.

# Question 13

☐ **13. Multiple Answer: Suppose that we are using a permutati…** Points: 2

| | |
|---|---|
| Question | Suppose that we are using a permutation representation and have the following individual: (1, 2, 3, 4, 5, 6, 7, 8, 9). Which of the following individuals, if any, could result from applying swap mutation? (This is a multiple answer question.) |
| Answer | None. Swap mutation does not make sense for the given individual. |
| | (1, 5, 3, 4, 2, 6, 7, 8, 9) |
| | (1, 5, 3, 4, 2, 7, 6, 8, 9) |
| | (1, 2, 3, 4, 5, 7, 6, 8, 9) |
| | (1, 2, 3, 4, 5, 5, 7, 8, 9) |
| | (9, 2, 3, 4, 5, 6, 7, 1, 8) |

# Question 13

**13. Multiple Answer: Suppose that we are using a permutati...**

Points: 2

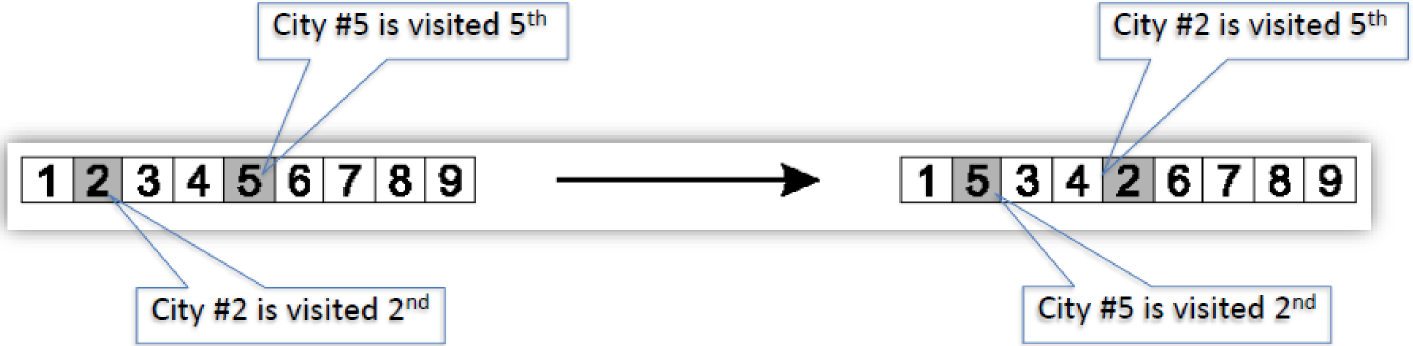
| | |
|---|---|
| **Question** | Suppose that we are using a permutation representation and have the following individual: (1, 2, 3, 4, 5, 6, 7, 8, 9). Which of the following individuals, if any, could result from applying swap mutation? (This is a multiple answer question.) |
| **Answer** | None. Swap mutation does not make sense for the given individual. |
| | ✓ (1, 5, 3, 4, 2, 6, 7, 8, 9) |
| | (1, 5, 3, 4, 2, 7, 6, 8, 9) |
| | ✓ (1, 2, 3, 4, 5, 7, 6, 8, 9) |
| | (1, 2, 3, 4, 5, 5, 7, 8, 9) |
| | (9, 2, 3, 4, 5, 6, 7, 1, 8) |

# Permutation Representations: Mutation

- Normal mutation operators lead to inadmissible solutions
  - e.g., bit-wise mutation: assume that gene *i* has value *j*
  - changing to some other value *k* would mean that *k* occurred twice and *j* no longer occurred
- Therefore mutation must change at least two values
- Mutation parameter now reflects the probability that some operator is applied once to the whole string, rather than individually in each position

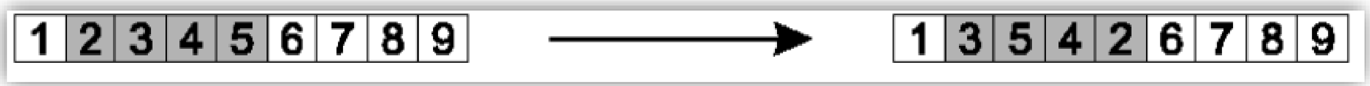
# Permutation Representations: Swap mutation

- Pick two alleles at random and swap their positions



# Permutation Representations: Scramble mutation

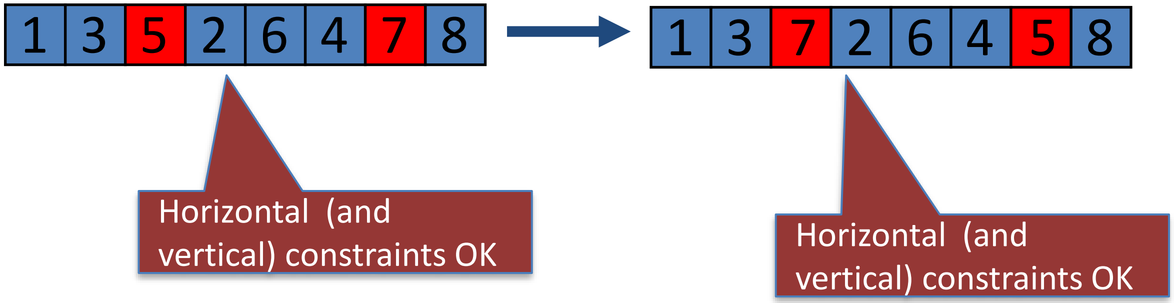
- Pick a subset of genes at random
- Randomly rearrange the alleles in those positions



# The 8-queens problem: Mutation

## Small variation in one permutation, e.g.:

- swapping values of two randomly chosen positions,



# Question 14

☐ **14. Multiple Answer: Suppose that we are using a permutati...**

Points: 2

| | |
|---|---|
| **Question** | Suppose that we are using a permutation representation and have the following individual: (1, 2, 3, 4, 5, 6, 7, 8, 9). Which of the following individuals, if any, could result from applying scramble mutation? (This is a multiple answer question.) |
| **Answer** | (1, 2, 3, 5, 2, 6, 7, 8, 8) |
| | (9, 8, 7, 6, 5, 4, 3, 2, 1) |
| | (1, 3, 5, 4, 2, 6, 7, 8, 9) |
| | (1, 3, 5, 5, 3, 6, 7, 8, 9) |
| | (2, 1, 3, 4, 6, 5, 7, 8 , 9) |
| | (1, 2, 3, 4, 5, 6, 9, 8, 7) |

# Question 14

☐ **14. Multiple Answer: Suppose that we are using a permutati...** Points: 2

| | |
|---|---|
| **Question** | Suppose that we are using a permutation representation and have the following individual: (1, 2, 3, 4, 5, 6, 7, 8, 9). Which of the following individuals, if any, could result from applying scramble mutation? (This is a multiple answer question.) |
| **Answer** | (1, 2, 3, 5, 2, 6, 7, 8, 8) |
| | (9, 8, 7, 6, 5, 4, 3, 2, 1) |
| | ✔ (1, 3, 5, 4, 2, 6, 7, 8, 9) |
| | (1, 3, 5, 5, 3, 6, 7, 8, 9) |
| | (2, 1, 3, 4, 6, 5, 7, 8 , 9) |
| | ✔ (1, 2, 3, 4, 5, 6, 9, 8, 7) |

# Permutation Representations: Scramble mutation

- Pick a subset of genes at random
- Randomly rearrange the alleles in those positions

# Question 15

**15. Multiple Answer: We are interested in problems that ma...**

Points: 2

| | |
|---|---|
| **Question** | We are interested in problems that may or may not have fitness functions and may or may not have constraints. In such problems, in which of the following settings is it natural to use an evolutionary algorithm? (This is a multiple answer question.) |
| **Answer** | In no such setting. |
| | In the constrained optimization setting, where we handle both constraints and a fitness function during the operation of the EA. |
| | In the constraint satisfaction setting, where we handle constraints during the operation of the EA but there is no fitness function. |
| | In the free optimization setting, where we only handle a fitness function during the EA's opration. Constraints, if any, are taken care of and integrated into the fitness function in a preprocessing step. |
| | In the free optimization setting, where we only handle a fitness function during the EA's opration. Constraints, if any, are disregarded in a preprocessing step. |

# Question 15

**15. Multiple Answer: We are interested in problems that ma...**

Points: 2

| | |
|---|---|
| Question | We are interested in problems that may or may not have fitness functions and may or may not have constraints. In such problems, in which of the following settings is it natural to use an evolutionary algorithm? (This is a multiple answer question.) |
| Answer | In no such setting. |
| | ✔ In the constrained optimization setting, where we handle both constraints and a fitness function during the operation of the EA. |
| | In the constraint satisfaction setting, where we handle constraints during the operation of the EA but there is no fitness function. |
| | ✔ In the free optimization setting, where we only handle a fitness function during the EA's opration. Constraints, if any, are taken care of and integrated into the fitness function in a preprocessing step. |
| | In the free optimization setting, where we only handle a fitness function during the EA's opration. Constraints, if any, are disregarded in a preprocessing step. |

# Constraint Handling (1/2)

Different ways of handling constraints in Evolutionary Algorithms

*Case 2 – (c), (d), and (e).* End up with COP: online (perhaps also offline) constraint handling + use constraint-handling EA online.

| | Objective Function | |
|---|---|---|
| **Constraints** | Yes | No |
| **Yes** | (e) (d) Constrained Optimization Problem (COP) (c) | Constraint Satisfaction Problem (CSP) |
| **No** | (a) Free Optimization Problem (FOP) (b) | Beyond our scope |

*Case 1 – (a) and (b).* End up with FOP: offline constraint handling + use standard EA online.

# Constraint Handling (2/2)

‘Constraint handling’ interpreted as ‘constraint transformation’

Case 1: CSP → **FOP**

All constraints are handled **indirectly**, i.e., $\Phi$ is transformed into f and later they are solved by \`simply‘ optimizing in $\langle S, f, \bullet \rangle$

Case 2: CSP → **COP**

Some constraints handled **indirectly** (those transformed into f)

Some constraints handled **directly** (those remaining constraints in $\psi$)

In the latter case we also have ‘constraint handling’ in the sense of ‘treated during the evolutionary search’

# Question 16

☐ **16. Multiple Answer: We consider in this question the pare…**

Points: 2

| | |
|---|---|
| **Question** | We consider in this question the parent selection step in evolutionary algorithms. Which of the following selection methods rely on knowledge of the entire population prior to selection? An example of " knowledge of the entire population" would be knowing the fitness of all individuals in the whole population, even if the population is distributed across multiple computers or multiple computational processes. (This is a multiple answer question.) |
| **Answer** | Selection in the island model<br>Fitness proportional selection<br>Tournament selection<br>Fitness sharing<br>Ranking selection |

# Question 16

☐ **16. Multiple Answer: We consider in this question the pare...** Points: 2

| | | |
|---|---|---|
| Question | We consider in this question the parent selection step in evolutionary algorithms. Which of the following selection methods rely on knowledge of the entire population prior to selection? An example of " knowledge of the entire population" would be knowing the fitness of all individuals in the whole population, even if the population is distributed across multiple computers or multiple computational processes. (This is a multiple answer question.) | |
| Answer | Selection in the island model | We looked at this already. |
| | ✓ Fitness proportional selection | Next slides. |
| | Tournament selection | Next slides. |
| | ✓ Fitness sharing | Next slides. |
| | ✓ Ranking selection | Next slides. |

# Parent Selection:
# Fitness-Proportionate Selection (FPS)

- Probability for individual *i* to be selected for mating in a population size *μ* with FPS is

$$P_{FPS}(i) = f_i \Big/ \sum_{j=1}^{\mu} f_j$$

- Limitations of FPS include:
  1. One highly fit member can rapidly take over if rest of population is much less fit: Premature Convergence
  2. At end of runs when fitnesses are similar, loss of selection pressure
  3. Highly susceptible to function transposition
- Scaling raw fitness *f(i)* to scaled fitness *f'(i)* can fix last two problems
  - Scaling by *windowing*:

$$f'(i) = f(i) - \beta^t$$

  where $\beta$ is a function (e.g., average) of fitness over last *t* generations
  - *Sigma scaling* normalizes fitness, taking population's mean fitness and standard deviation into consideration

# Parent Selection: Rank-Based Selection (RBS)

- Attempt to remove problems of FPS by basing selection probabilities on
  - *Relative* – according to rank in population – rather than
  - *Absolute* fitness $P_{FPS}(i)$
- Idea of RBS: Rank population according to fitness and then base selection probabilities on "rank" (or "prize")
  - Fittest individual has rank or prize $\mu$-*1* (or $\mu$) and
  - Worst fit individual has rank or prize 0 (or *1*)
- This imposes a sorting overhead on the algorithm, but this is usually negligible compared to the fitness evaluation time
  - However, fitness evaluation time varies dramatically from application to application

# Explicit Approaches for Preserving Diversity: Fitness Sharing (1/2)

- Restricts the number of individuals within a given niche by "sharing" their fitness:
  - Niche fitness is a shared resource – think sheep eating grass
  - Individuals are allocated to a niche in proportion to the niche fitness
- Need to set the size of the niche $\sigma_{share}$ in either genotype or phenotype space
  - Here we assume genotype space
- Run EA as normal but after each generation set

$$f'(i) = \frac{f(i)}{\sum_{j=1}^{\mu} sh(d(i,j))} \qquad sh(d) = \begin{cases} 1-\left(d/\sigma_{\text{share}}\right)^{\alpha} & d \le \sigma_{\text{share}} \\ 0 & otherwise \end{cases}$$

# Parent Selection:
# Global versus Local Computation for Selection

- All methods above rely on *global* population statistics
  - In other words, the fitness of all individuals need to be known in order to compute the fitness of a single individual
  - *Issue 1*: How realistic is the computation of global population statistics in the natural world (ref. bio-inspired AI)?
  - *Issue 2*: Such computation could be a bottleneck, esp., on parallel machines, with very large populations

- Is there a way to avoid *global* computation of population statistics?
  - Yes, there are *local* approaches, for example tournament selection

# Parent Selection: Tournament Selection

- Idea for a procedure – Tournament Selection – using only local fitness information:
  - Pick *k* members  at random, then select the best of these
  - Repeat to select more individuals

- Probability of selecting individual *i*  will depend on:
  - Rank of *i*, i.e., how fit the individual is
  - Size of sample *k*
    - higher *k* increases selection pressure
  - Whether contestants are picked with replacement
    - Picking without replacement increases selection pressure
  - Whether fittest contestant always wins (deterministic) or this happens with probability *p*

# Question 17

☐ **17. Multiple Answer: A problem instance can be solved by s...**

Points: 2

| | |
|---|---|
| **Question** | A problem instance can be solved by some algorithm (with no restrictions on running time) and any solution can be verified by another algorithm in polynomial time. Which complexity class or classes does this describe? (This is a multiple answer question.) |
| **Answer** | No complexity class is described in this way. |
| | This complexity class exists, but it is not among the ones listed below. |
| | Class P |
| | Class NP |
| | Class NP-complete |
| | Class NP-hard |

# Question 17

☐ **17. Multiple Answer: A problem instance can be solved by s...**

Points: 2

| | |
|---|---|
| **Question** | A problem instance can be solved by some algorithm (with no restrictions on running time) and any solution can be verified by another algorithm in polynomial time. Which complexity class or classes does this describe? (This is a multiple answer question.) |
| **Answer** | No complexity class is described in this way. |
| | This complexity class exists, but it is not among the ones listed below. |
| | Class P |
| | ✅ Class NP |
| | Class NP-complete |
| | Class NP-hard |

# The *P* and *NP* Complexity Classes

- Class *P*:
  - *P* for "polynomial time"
  - A problem belongs to *P* if there exists an algorithm that solves the problem that runs in polynomial time:
    - Polynomial time: $O(n)$, $O(n \log n)$, $O(n^2)$, …
    - Not polynomial time: $O(n^n)$, $O(2^n)$, …
  - Many "traditional" problems, e.g., searching and sorting, are in class *P*
- Class *NP*:
  - *NP* for "nondeterministic polynomial time"
  - A problem belongs to *NP* if there exists an algorithm *A* (no claims about its running time) whose problem solutions can be checked (or verified) in polynomial time by an algorithm *B*
  - Many "hard" computational problems from AI are in the class *NP*

# Question 18

☐ **18. Multiple Answer: Consider a GA with a population of th...**

Points: 3

| | |
|---|---|
| Question | Consider a GA with a population of these bistrings: 01101, 11000, 01000, and 10011. Further suppose that we have the following values for the fitness function f:<br>• f(01101) = 15<br>• f(11000) = 40<br>• f(01000) = 8<br>• f(10011) = 37<br>We assume that the population size is constant. What are the expected number of copies of individuals 01101 and 11000 after parent selection? (This is a multiple answer question.) |
| Answer | 0.08<br>0.15<br>0.32<br>0.37<br>0.4<br>0.6<br>1.48<br>1.6 |

# Question 18

☐ **18. Multiple Answer: Consider a GA with a population of th...** Points: 3

| | |
|---|---|
| Question | Consider a GA with a population of these bistrings: 01101, 11000, 01000, and 10011. Further suppose that we have the following values for the fitness function f:<br>• f(01101) = 15<br>• f(11000) = 40<br>• f(01000) = 8<br>• f(10011) = 37<br>We assume that the population size is constant. What are the expected number of copies of individuals 01101 and 11000 after parent selection? (This is a multiple answer question.) |
| Answer | 0.08<br>0.15<br>0.32<br>0.37<br>0.4<br>✔ 0.6<br>1.48<br>✔ 1.6 |

Total fitness: 15 + 40 + 8 + 37 = 100

Proportional fitness of 01101: 0.15

Proportional fitness of 11000: 0.4

Expected copies of 01101: 0.15 * 4 = 0.6

Expected copies of 11000: 0.4 * 4 = 1.6

# Genetic Algorithms:
# SGA technical summary tableau

| | |
|---|---|
| Representation | Bit-strings |
| Recombination | 1-Point crossover |
| Mutation | Bit flip |
| Parent selection | Fitness proportional – implemented by Roulette Wheel |
| Survivor selection | Generational |

# Genetic Algorithms: SGA reproduction cycle - Overview

- Select parents for the mating pool (size of mating pool = population size)
- Shuffle the mating pool
- Apply crossover for each consecutive pair with probability $p_c$, otherwise copy parents
- Apply mutation for each offspring (bit-flip with probability $p_m$ independently for each bit)
- Replace the whole population with the resulting offspring

See slides or E&S text book page 35:

| String no. | Initial population | $x$ Value | Fitness $f(x) = x^2$ | $Prob_i$ | Expected count | Actual count |
|---|---|---|---|---|---|---|
| 1 | 0 1 1 0 1 | 13 | 169 | 0.14 | 0.58 | 1 |
| 2 | 1 1 0 0 0 | 24 | 576 | 0.49 | 1.97 | 2 |
| 3 | 0 1 0 0 0 | 8 | 64 | 0.06 | 0.22 | 0 |
| 4 | 1 0 0 1 1 | 19 | 361 | 0.31 | 1.23 | 1 |
| Sum | | | 1170 | 1.00 | 4.00 | 4 |
| Average | | | 293 | 0.25 | 1.00 | 1 |
| Max | | | 576 | 0.49 | 1.97 | 2 |

**Table 3.1.** The $x^2$ example, 1: initialisation, evaluation, and parent selection

After having made the essential design decisions, we can execute a full selection–reproduction cycle. Table 3.1 shows a random initial population of four genotypes, the corresponding phenotypes, and their fitness values. The cycle then starts with selecting the parents to seed the next generation. The fourth column of Table 3.1 shows the expected number of copies of each individual after parent selection, being $f_i/\bar{f}$, where $\bar{f}$ denotes the average fitness (displayed values are rounded up). As can be seen, these numbers are not integers; rather they represent a probability distribution, and the mating pool is created by making random choices to sample from this distribution. The